SSL-PROJECT-REPORT
Snake Stack

Shiven Toshniwal Rishabh Iyer

May 2026

Contents

1 External libraries used with purpose **2**

2 Description of all features implemented **2**
2.1 Gameplay 3
2.2 Admin 4

3 Directory structure & how the 3 layers communicate **5**
3.1 Directory structure 5
3.2 How layers communicate 5
3.3 Format for history.txt 6
3.4 Important functions defined in various files 6
3.4.1 ui.js 6
3.4.2 game.js 6
3.4.3 api.js 7
3.4.4 app.py 7

4 Usage **7**

5 Hurdles faced and how we overcame them **7**

6 What we would do in 10 extra hours **8**

1 External libraries used with purpose

- **Bootstrap 5** front end framework - For Styling [1]
- **Math.random** - For random generation of food items on the grid [5]
- **Math.floor** - For random selection of coordinates as well as random selection of fruits [4]
- **Flask** - Provides web server and routing capabilities [2] [3]
- **datetime** - Formats dates for the history.txt log
- **os** - Handles directory paths for serving static files

2 Description of all features implemented

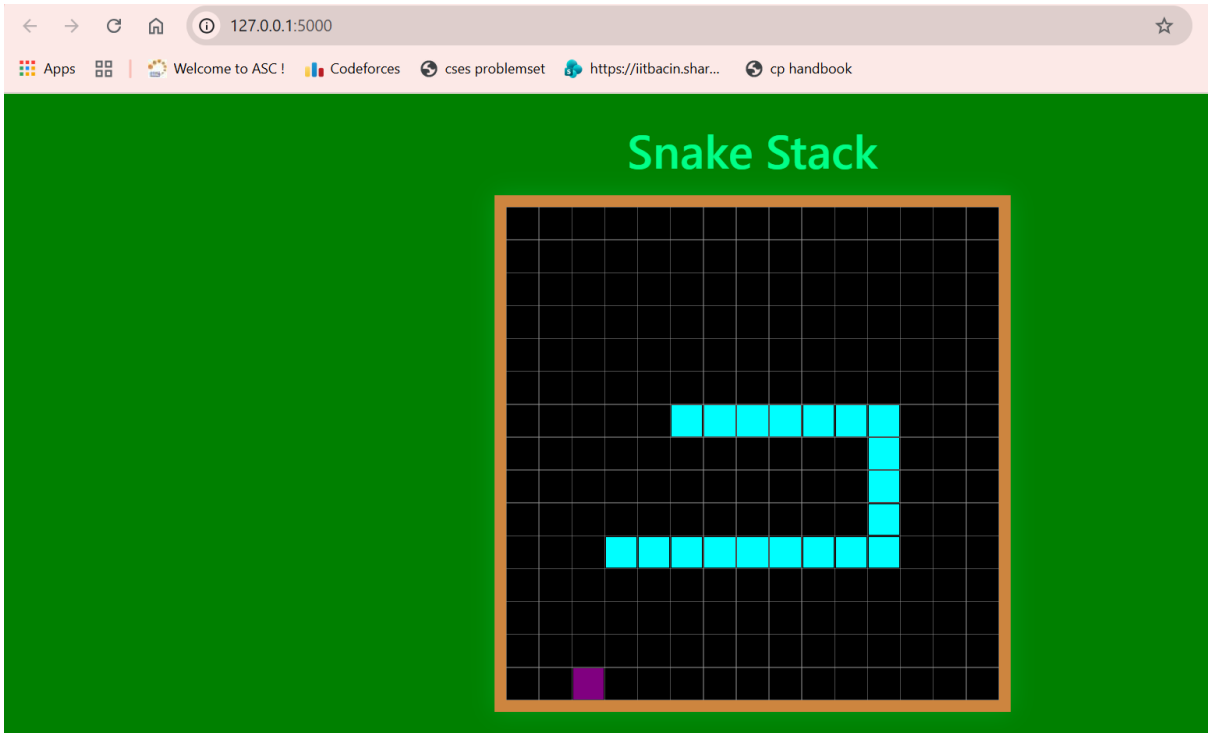


Figure 1: Ongoing game

2.1 Gameplay

- **Welcome modal** on starting game with start button and game information (simple styling with basic properties like font-color,background-color,font style,size and alignment) and asks for username which if not entered leads to display of alert message(checking done after removing whitespaces in username). (2a)

- Storing **snake** as an array of pairs of coordinates.(To keep track of position of snake body. Per move new element is added at start of snake array and last element is popped out, and remaining elements are kept unchanged)

- Taking **keyboard input** is used to specify direction of motion of snake .(snake cannot reverse direction in one move).Head value gets updated (Keyboard inputs taken by event listeners)

- Storing **food items** as array of pairs of coordinates.(Foods of have variables like name,value,color,immunity,speed).

- Every 200ms, the **boxes** corresponding to coordinates of snake and foods are **colored**.(So it appears that the snake is moving by seeing difference between succesive renderings)(1)

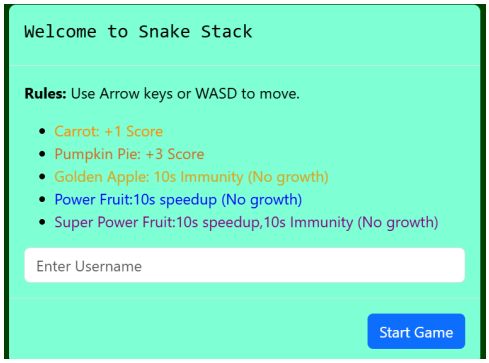
- **Snake dies** when the coordinates of head either coincide with another element of snake array or when it leaves the grid.(Head coordinates outside grid boundaries)

- 5 different **types of fruits** that the snake can feed on.(Different properties of foods like immunity,length growth and speedup) One food item instantly appears when the previous is eaten.(Only one food at a time is displayed and it is selected randomly from food array. Generation position is also random).

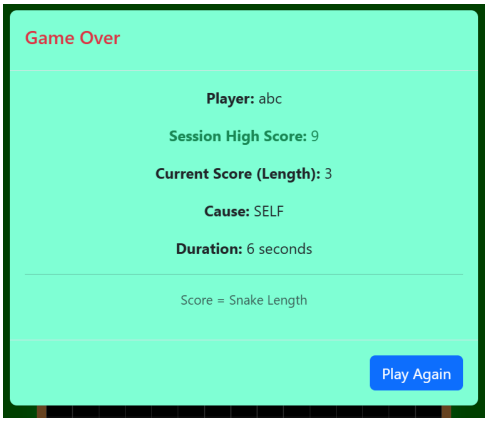
- **Increasing length** by adding a copy of last "food value" number of elements of snake array to the array, so when snake moves after next "food value" moves we can see those elements(growth of snake) that would otherwise have been popped out).

- Setting a 10s **immunity** timer in which the snake does not die on either hitting the wall or itself, if it has eaten a food that gives immunity.

- Setting a 10s **speed up** by reducing the time interval between successive renderings of the game(board,snake,food) temporarily, if snake eats a food that gives speed up.(So the gap in which click has to be done is reduced and gameplay becomes faster)



(a) Game Start



(b) Game Over

Figure 2: User Interface

- Displays game-play **statistics** of current session(username,length of snake,duration of play,cause of death,session high score) on a **game-over modal** and updates session high score if needed.(2b)

2.2 Admin

- **Checking history.txt** for existence/emptiness.

- Display **menu driven interface**

  1. Viewing **recent games' data** (that present in history.txt) sorted by timestamp(most recent first).

  2. Viewing **analytics** for games in history.txt.(Mean score, mean time, fraction of wall deaths, fraction of self deaths).

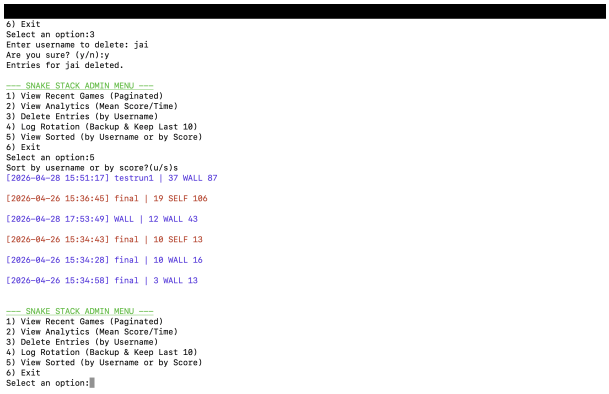
  3. **Deleting entries** by username from history.txt

  4. **Log rotation** in which history.txt is truncated to latest 10 entries and backup is created in the form of .tar.gz file.

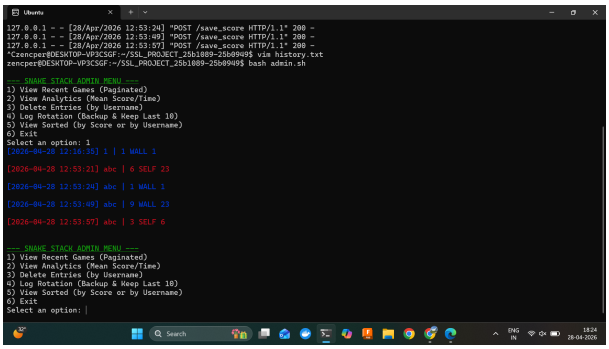
  5. **Sorting** by username or score(3a)

  6. **Exit** option

- **Handling invalid input** and displaying error message. (Input format is defined in prompt wherever needed)



(a) Sorting



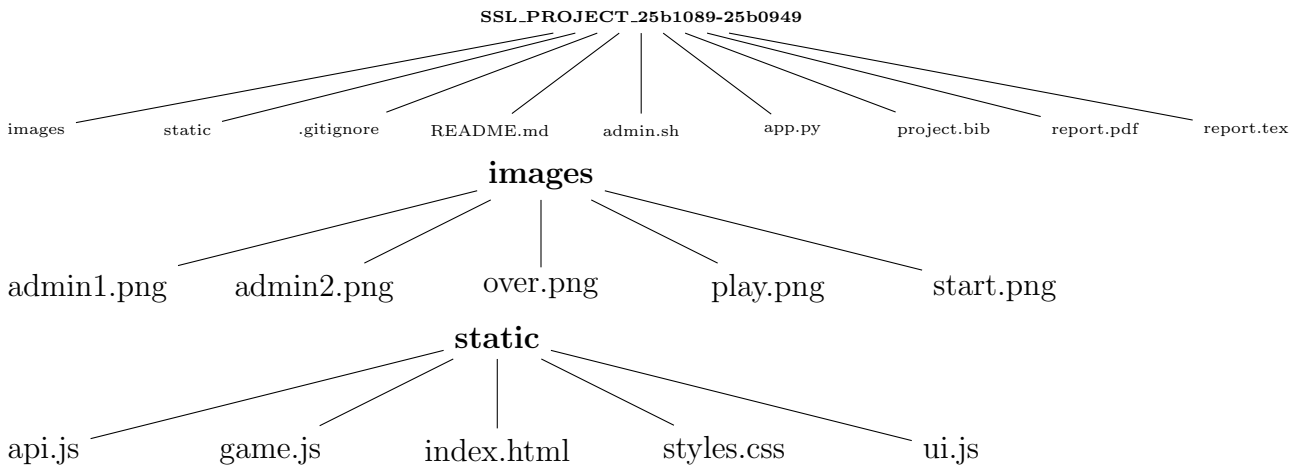
(b) Recent games

Figure 3: Admin Features

- Using **ANSI escape codes** for colored/formatted output.(Wall deaths in blue, self deaths in red, error messages in bold)
  (3b)

3 Directory structure & how the 3 layers communicate

3.1 Directory structure



Also a Makefile exists in SSL_PROJECT_25b1089-25b0949

3.2 How layers communicate

- At first, the user interacts with JavaScript front end framework. When the game ends, the front end sends the final score to Flask Server using fetch API.

- Request is POST with a JSON body. Inside app.py, the score is parsed, validated and appended to history.txt with timestamp.

- Managing history.txt and backup files is done by admin.sh. We have used bash commands like sed, awk, grep, cut, sort and case to create a menu driven interface with different options.

3.3 Format for history.txt

[timestamp] name | score cause duration

Chosen because with this format, sorting becomes easier and since game data is space separated, it can be handled by awk.

3.4 Important functions defined in various files

3.4.1 ui.js

- **init()**: For checking username

- **showGameOver(score,cause,duration)**: For showing Game Over modal.

3.4.2 game.js

- **init()**:Initializing snake,food and calling handleInput() (for keyboard input),draw() (initial grid) and setting interval for loop() (200 ms).

- **spawnFood()**:Defining food types and ensuring food generates where snake is not there.

- **loop()**:Updating position of snake head and checking if snake is dead(and cause of death).Calling speedUp() and startImmunity() if needed. Increasing snake length if needed. Calling draw() for rendering.

- **speedUp()**: Increasing snake speed for 10s by reducing the time between renderings.

- **startImmunity()**: Setting immunity timer for 10s.

- **draw()**: Drawing grid and displaying snake and fruits(by coloring corresponding positions).

- **handleInput()**:Updates direction of motion of snake as per keyboard commands(Arrow keys or WASD).

- **endGame(cause)**: Terminating running interval loops and calling showGameOver in ui.js, saveScore in api.js.

3.4.3 api.js

- **saveScore(gameData)**:Sends score using fetch API

3.4.4 app.py

- **index**: Serves frontend to the browser.

- **save_score**: Validates received game data and appends it to history.txt in the decided format.

4 Usage

- Running game on server

  1. Type 'python3 app.py' on command line.

  2. Navigate to http://127.0.0.1:5000 on browser.

  3. Enter username and press Start Game.

  4. Can use arrow keys or WASD for snake motion.

  5. Press Play Again to restart if game over

- Running admin.sh

  1. Type 'bash admin.sh' on command line.

  2. Navigate menu by entering number corresponding to required option.

  3. Where valid, enter required suboptions out of those visible

5 Hurdles faced and how we overcame them

- Adjusting the display size of the grid and modal - By trial and error.

- Adjusting timers for changing speed of snake and setting immunity - Disabling existing loop and setting timer for running a new loop temporarily.

- Ensuring proper parsing and sorting of data in history.txt. For example, if username matches WALL or SELF - By using proper delimiters as per history.txt's format.

- Ensuring that a fruit does not generate on snake's body - Keeping in mind that the randomly generated fruit appears on the grid excluding all the coordinates inside snake array.

6 What we would do in 10 extra hours

- Improve the graphics of both the snake and fruits.

- An option for the user to vary speed of snake, vary the types and numbers of fruits generated at a time.

References

[1] The Bootstrap Authors. Bootstrap5. https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css.

[2] *Flask documentation*. https://flask.palletsprojects.com/en/stable/.

[3] Real Python. Flask tutorials. https://realpython.com/tutorials/flask/.

[4] W3Schools. Js math floor() method. https://www.w3schools.com/jsref/jsref_floor.asp.

[5] W3Schools. The js random. https://www.w3schools.com/js/js_random.asp.